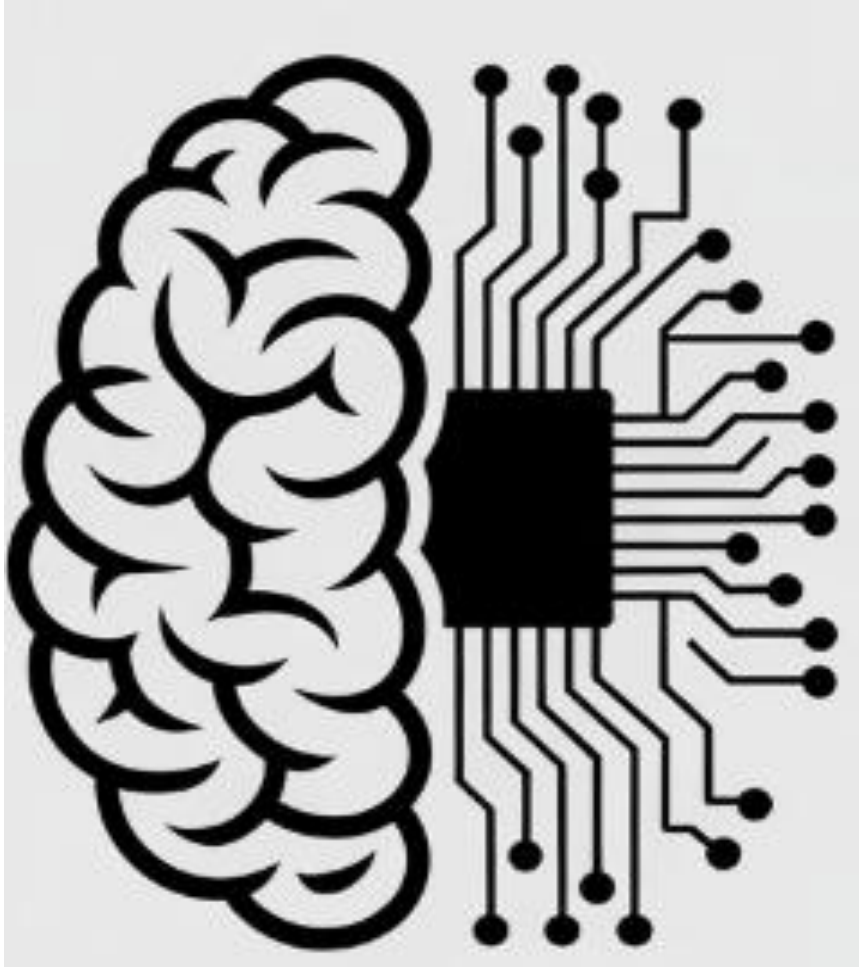
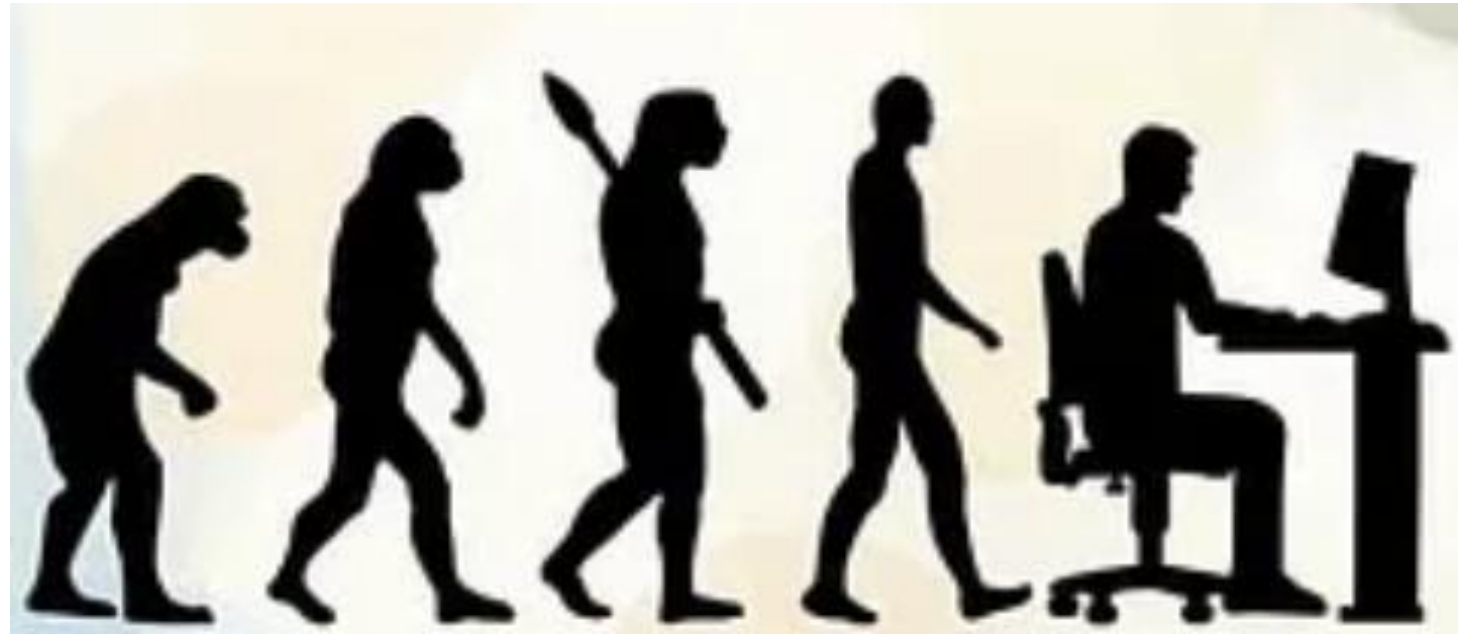




Evolutionary Search: Genetic Algorithm

Dr. Sobia Tariq Javed

What is Evolutionary Computation?



- **Evolutionary Computation (EC)** is a branch of **artificial intelligence**.
- It uses **mechanisms inspired by biological evolution** to solve optimization and search problems.
- It involves algorithms that mimic the process of natural evolution to evolve solutions to complex problems.

What is EC?

- Darwin's principle of "Survival of the fittest" can be used as a starting point in introducing evolutionary computation.
- EC deals with a range of problem-solving techniques based on principles of biological evolution, such as natural selection and genetic inheritance.

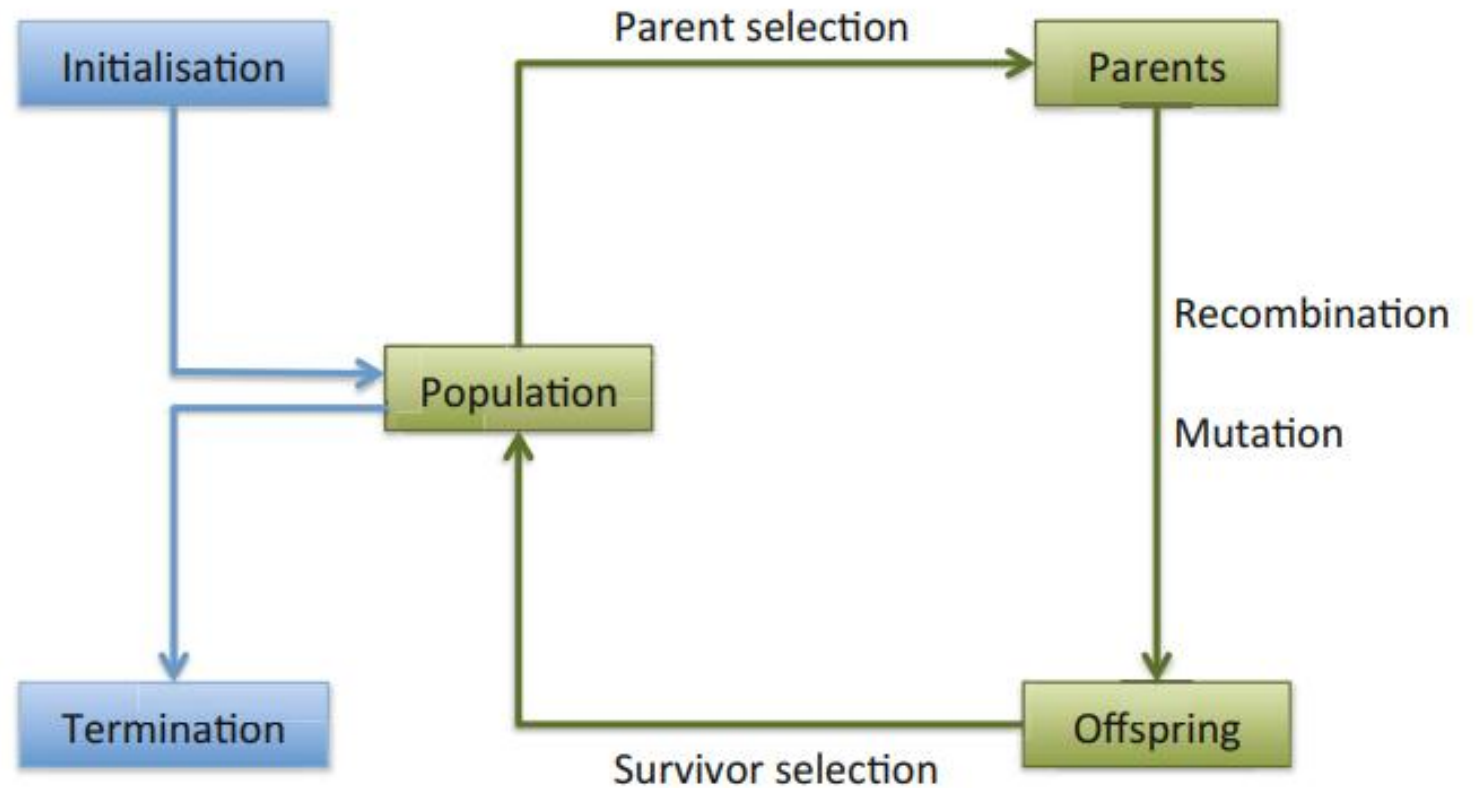


Evolution- Racehorse Bloodlines

- Racehorse competition
- **Goal:** Run FAST!
- **Solution:** Champion Horses (DNA)
- **Optimization :** Breeding history
- Racehorse's origin from 3 horses
 - Byerley Turk (1680-1703 AD)
 - Darley Arabian (1700-1722 AD)
 - Godolphin Arabian (1724-1753 AD)
- Select good horses (winners & relatives)
- Breed
- next generation of horses
- competitions



The general scheme of an evolutionary algorithm as a flowchart



Genetic Algorithms

- John Holland, 1960s
- Based on ideas from Darwinian Evolution
- Can be used to solve a variety of problems that are not easy to solve using other techniques
- “Adaptation in natural and artificial systems”, 1975.



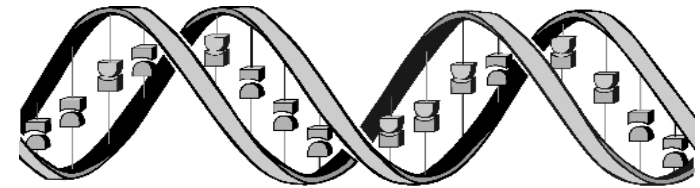
Principle of Natural Selection

- Giraffes with slightly longer necks could feed on leaves of higher branches when all lower ones had been eaten off.
- They had a better chance of survival.
- Favorable characteristic propagated through generations of giraffes.
- Now, evolved species has long necks.

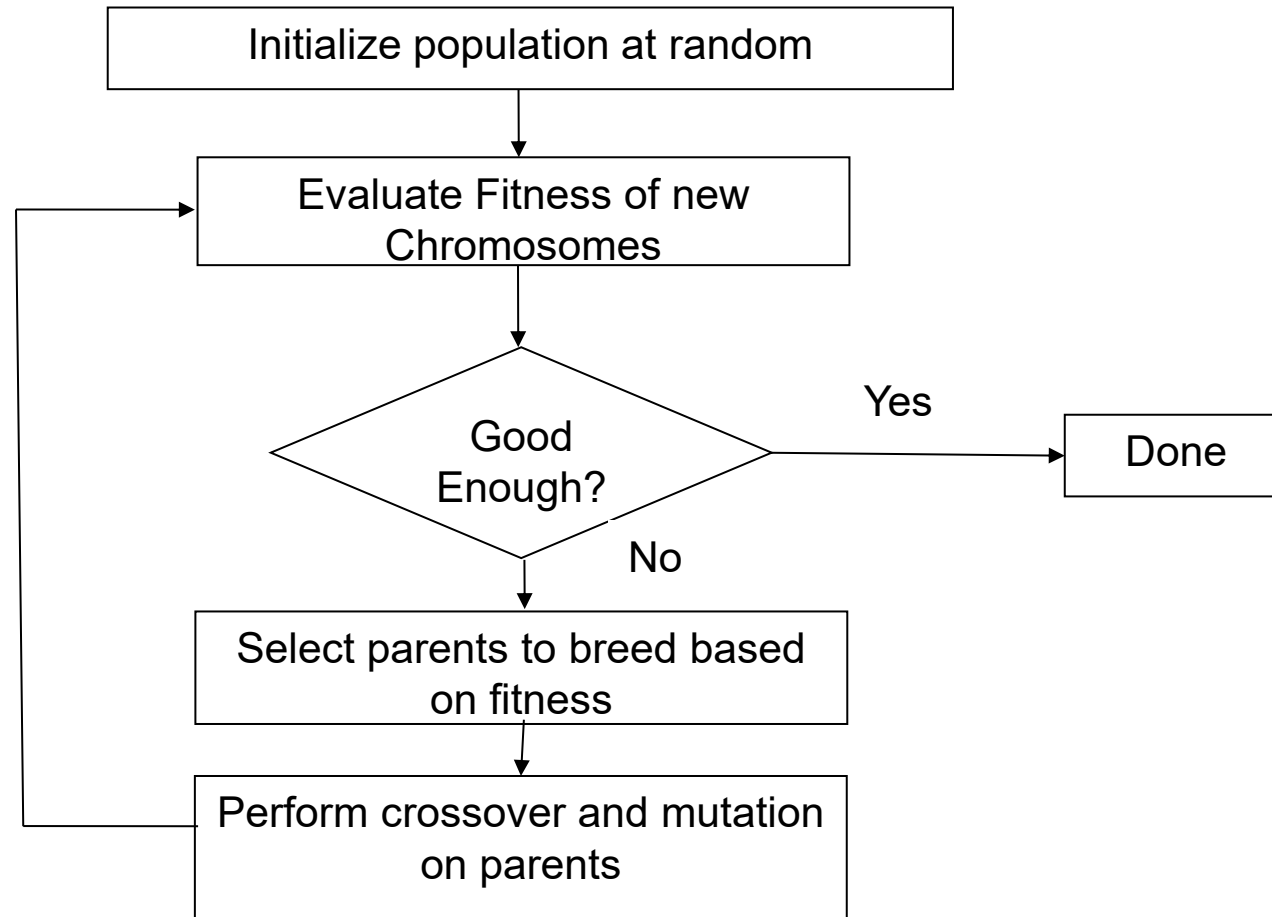


Genetic Algorithms

- Problems are not solved through logical reasoning alone.
- Instead, populations of competing candidate solutions are created.
- These solutions evolve over generations, becoming progressively better.
- The evolution process mimics biological evolution:
 - **Selection:** Choosing the fittest candidates.
 - **Mutation:** Introducing random variations.
 - **Reproduction:** Combining traits of successful candidates.
- Survival of the Fittest:
 - Less effective candidates are eliminated.
 - Promising candidates survive and reproduce.
- This approach refines solutions iteratively toward optimal or near-optimal outcomes.



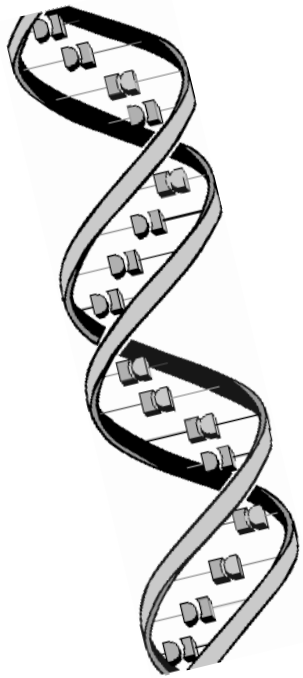
Basic Flow of GA



Basics

- **Gene:** The basic unit representing a single characteristic or feature of an individual (like eye color). The specific value assigned to a gene is called an allele.
- **Chromosome:** A sequence of genes that collectively represents an individual or a possible solution to the problem. Each chromosome corresponds to a point in the search space.
- **Population:** A group of chromosomes that forms the entire set of candidate solutions in a given generation.
- **Importance of Representation:** Choosing an appropriate chromosome representation is crucial for the efficiency and complexity of the genetic algorithm (GA), as it impacts the algorithm's ability to explore and exploit the search space effectively.

Genotype vs Phenotype



Genotype

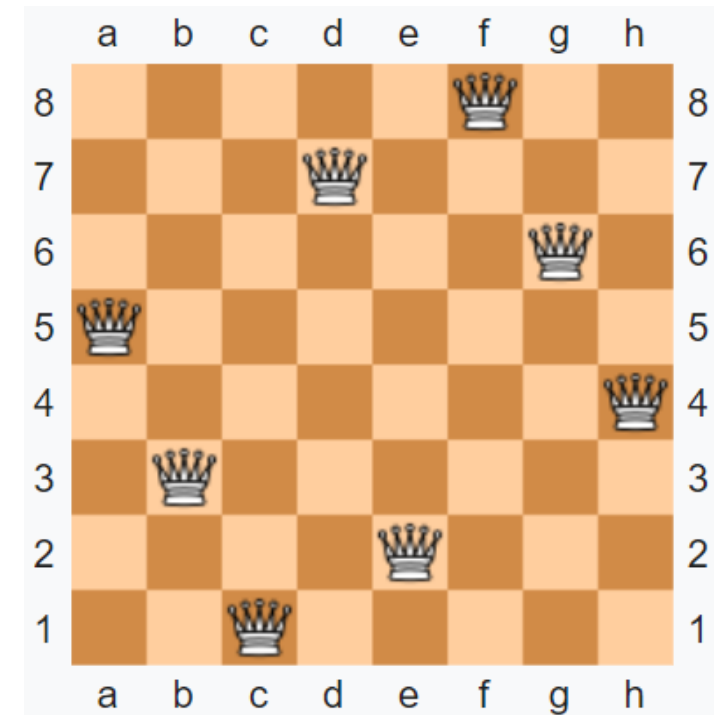
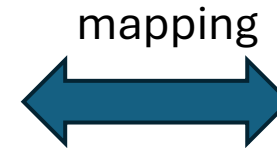


Phenotype

The 8 Queen Problem: Representation



Genotype: a permutation
of the numbers 1 - 8

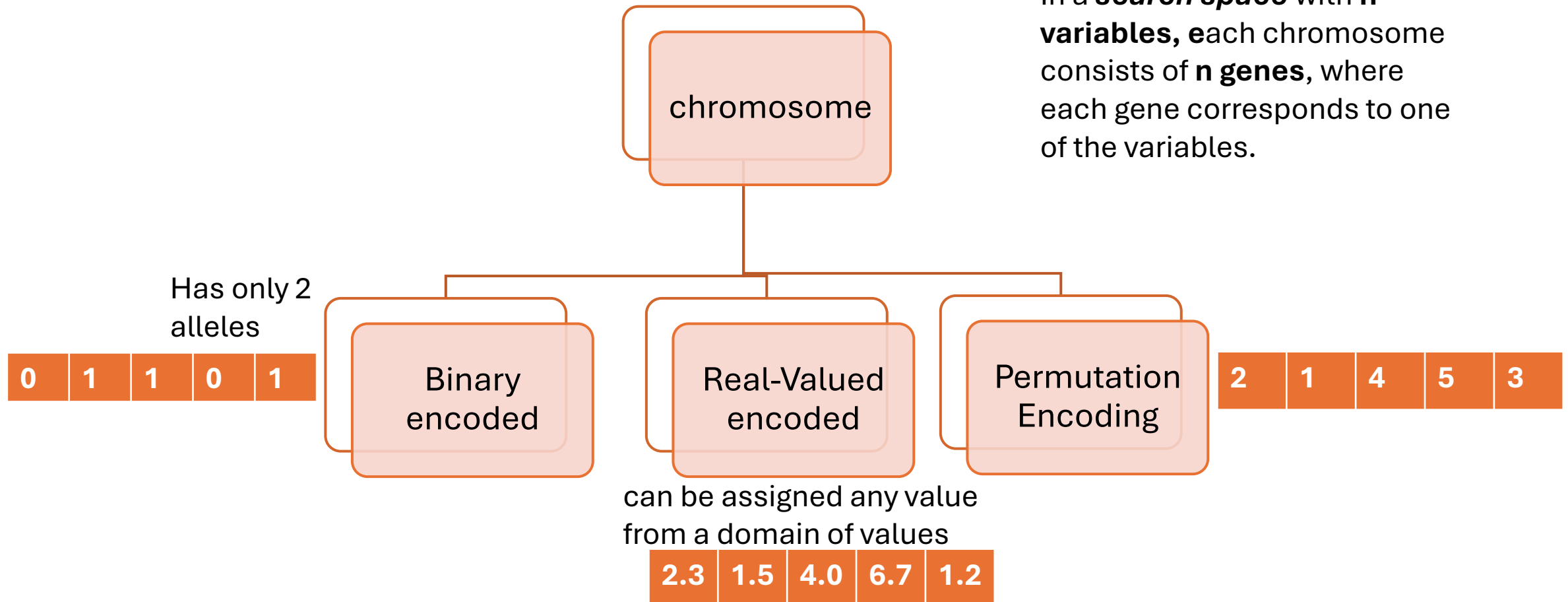


Phenotype: a board configuration

Chromosome

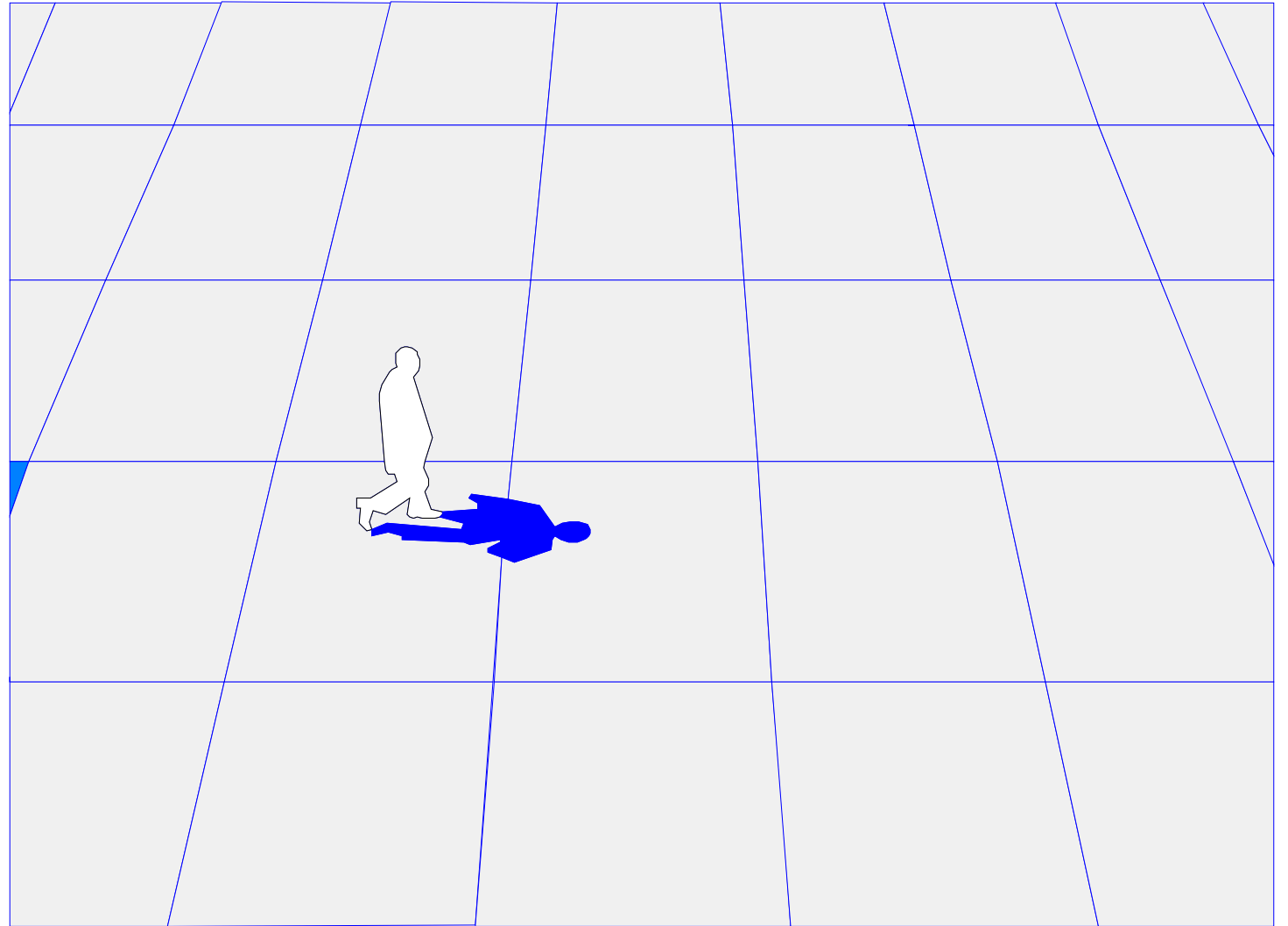
- Chromosome is a vector of fixed length.

In a **search space** with **n variables**, each chromosome consists of **n genes**, where each gene corresponds to one of the variables.



Genetic Algorithms

- *“Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, solutions you might not otherwise find in a lifetime.”*
- - Salvatore Mangano
- *Computer Design*, May 1995



Chromosome Design for Personal Finance Management Problem

- Design a chromosome for Personal Finance Management Problem. You need to optimize how to allocate your monthly income to different expenses (e.g.,¹rent,²groceries,³savings,⁴entertainment) while maximizing savings and minimizing overspending. Minimum you can allocated \$10 and maximum \$1000.
- Chromosome Representation:
 - Each **gene** in the chromosome could represent the amount of money allocated to a specific expense.
 - For example, a chromosome could be represented as `[800, 200, 150, 50]` indicating allocations of \$800 for rent, \$200 for groceries, \$150 for savings, and \$50 for entertainment.

Chromosome for Clothing Combination for the Day.

- Design a chromosome for Clothing Combination for the Day. You want to decide what clothes (a jacket, a hat, tie, gloves, and long shoes.) to wear today based on weather conditions, personal preferences, and style.
 - Chromosome Representation:
 - Each clothing item is represented by a **gene**, where 1 means the item is chosen to wear, and 0 means it is excluded.
 - For example, a chromosome `[1, 0, 1, 0, 1]` might represent wearing a jacket, skipping a hat, and including pants and shoes.

Evaluation (Fitness Function)

- **Purpose:** Quantifies the optimality of a solution to rank it against others.
- **Measure of Performance:** Indicates how close the solution is to the desired output/Goal.
- **Evaluation Process:**
 - Decodes a chromosome representation.
 - Assigns a fitness measure to the solution.
- **Problem-Specific:** The fitness function depends on the specific problem being solved.

Initial Population

- **Population Generation:**

- The evolutionary process begins by generating an initial population.

- **Standard Method:**

- Gene values are randomly selected to ensure uniform representation of the search space.

- **Heuristic Initialization:**

- If prior knowledge about the problem is available, heuristics can bias the population toward good solutions.
- **Risk:** Heuristics may lead to premature convergence to a local optimum, as not all parts of the search space are explored.

Initial Population-Impact of Population Size

Small Population:

- Covers a small search space.
- Requires more generations to find a solution but has lower variance.

Large Population:

- Covers a large search space.
- Requires fewer generations to find a solution but increases time and variance.

Mutation to Compensate:

For small populations, increasing the mutation rate can force exploration of a larger search space.

Selection Operators

- Individuals are chosen based on fitness to form an intermediate population for:
 - **Survival:** Some remain unchanged.
 - **Mutation or Replacement:** Some are modified or replaced.
 - **Crossover and Mutation:** Offspring replace parents after genetic operations.



Selection Operators

- **Selection pressure**

the degree to which better-performing individuals are favored over less-fit ones

- **High Selection Pressure:**

- Strong focus on fitter individuals.
- Faster convergence but increased risk of premature convergence to a local optimum.

- **Low Selection Pressure:**

- Slower convergence.
- Better exploration of the search space, reducing the risk of local optima.

Selection Operators

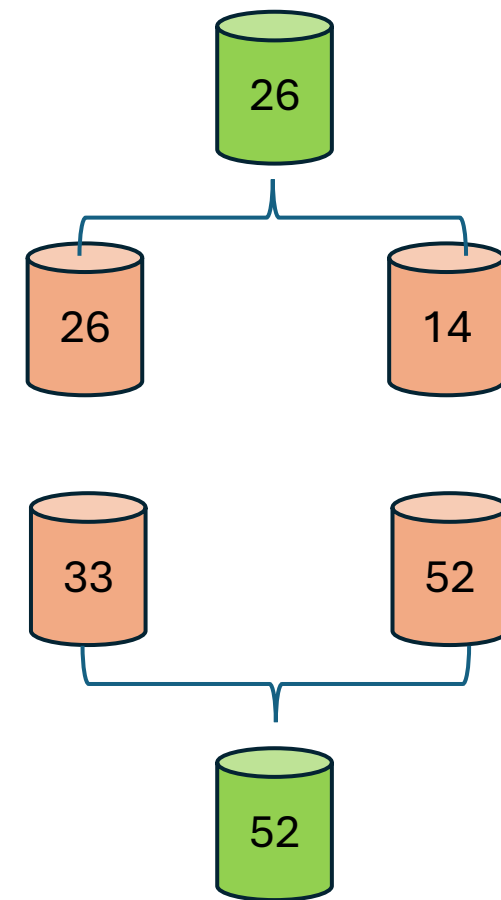
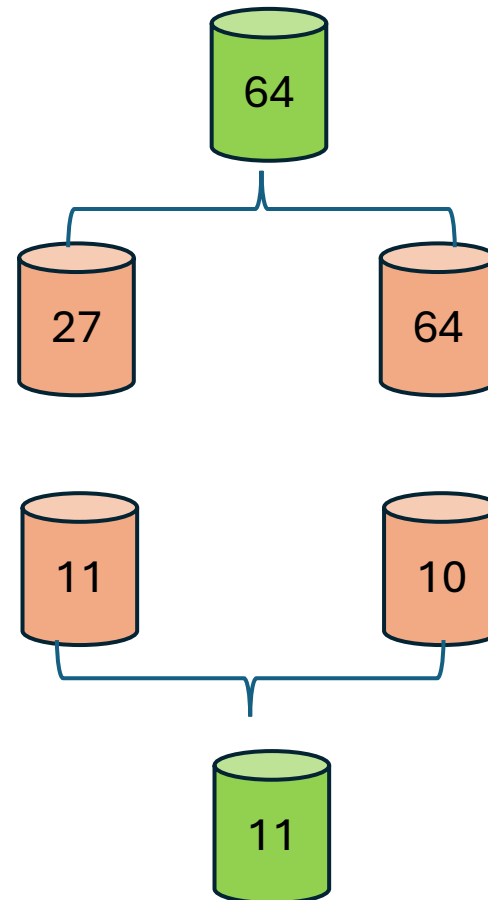
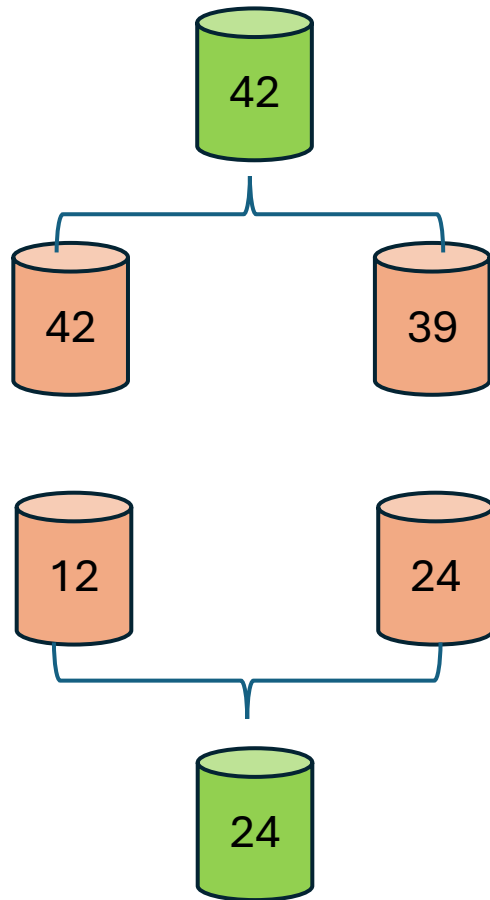
- Different selection schemes
 - Tournament selection
 - Roulette wheel selection
 - Rank-based selection
 - Random selection

Tournament selection

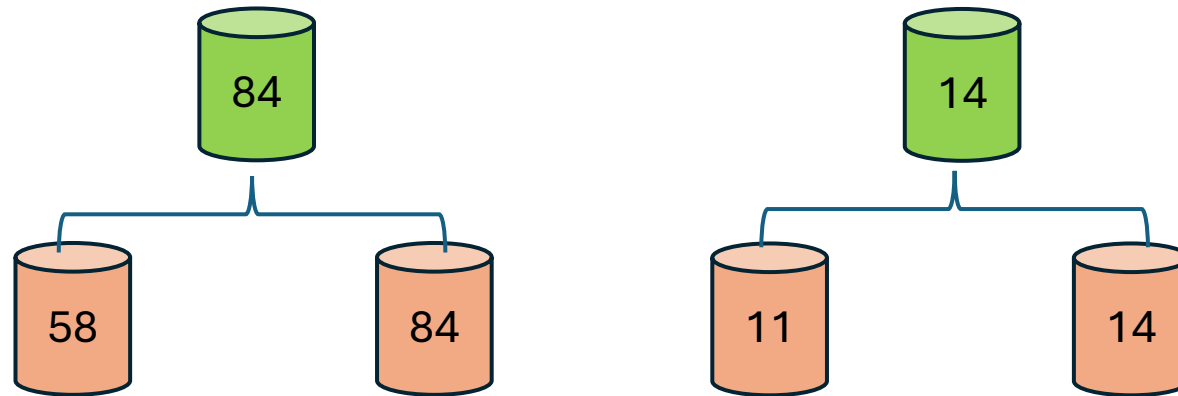
- **Process:**
 - Multiple tournaments are conducted among randomly chosen individuals from the population.
 - The winner of each tournament is selected for the next generation.
- **Adjusting Selection Pressure-Tournament Size:**
 - **Larger size** Favors stronger individuals, increasing selection pressure.
 - **Smaller size** Provides weaker individuals a higher chance of selection, reducing selection pressure.



Tournament selection

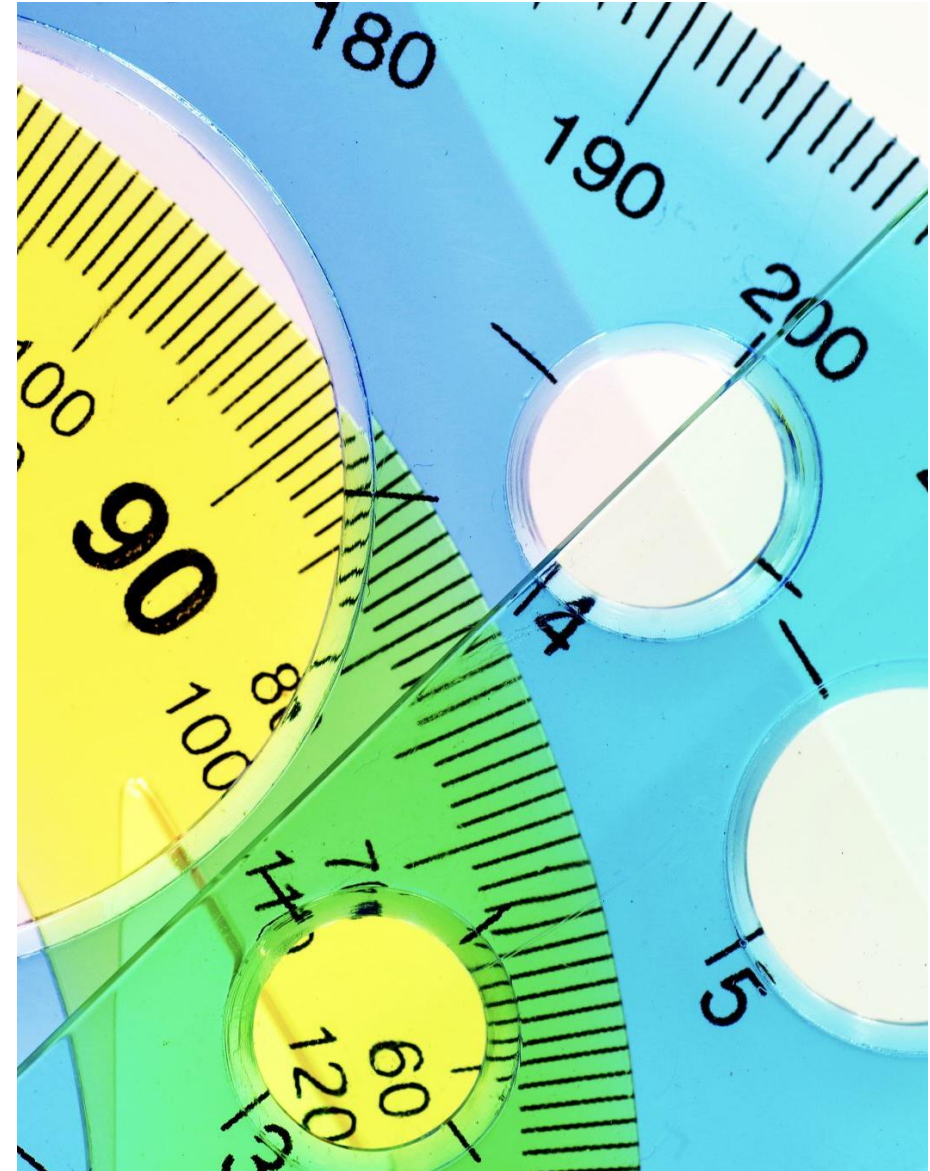


Tournament selection



Roulette wheel selection

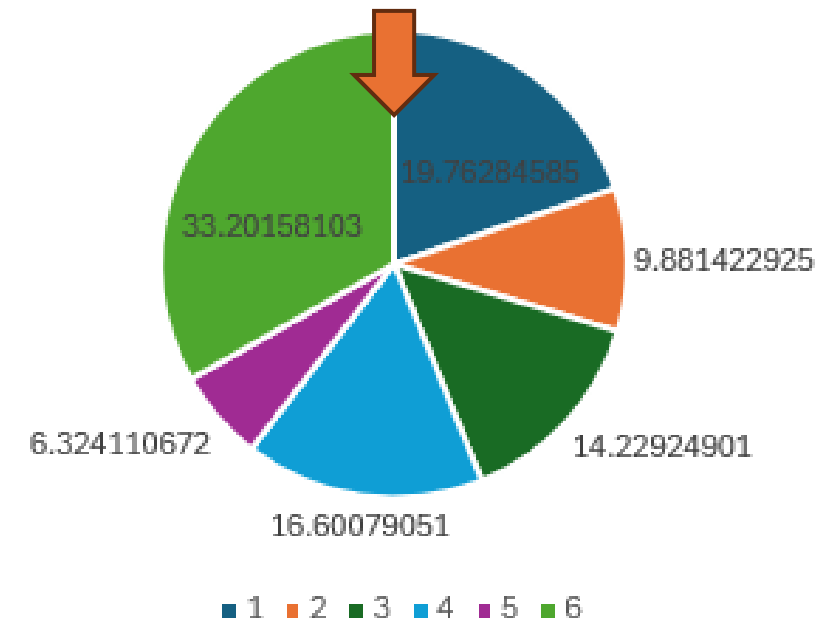
- Roulette Wheel Selection selects individuals from a population based on their fitness values, where those with higher fitness have a greater likelihood of being chosen.



Roulette wheel /Proportionate selection

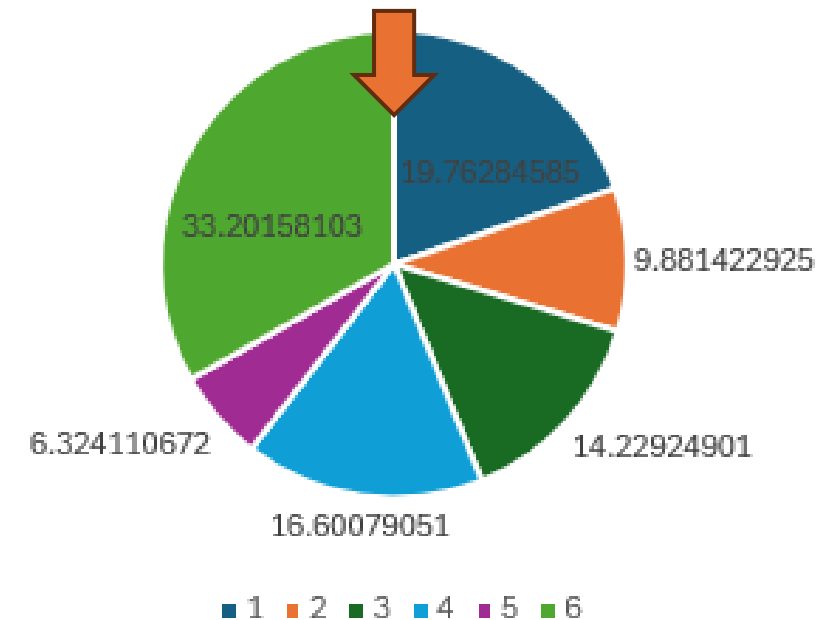
Maximization Problem

Chromosome #	Fitness	% of RW	range
A	50		
B	25		
C	36		
D	42		
E	16		
F	84		
Total			



Roulette wheel /Proportionate selection

Chromosome #	Fitness	% of RW	range
A	50	19.76	0-19.76
B	25	9.88	19.76-29.64
C	36	14.23	29.64-43.87
D	42	16.60	43.87-60.47
E	16	6.32	60.47-66.8
F	84	33.20	66.8-100
Total	253	100	



Rank-based selection-Method 1

- Rank-based selection assigns selection probabilities to individuals based on their rank rather than their raw fitness values.
- **Advantage:** This helps to avoid excessive selection pressure caused by large fitness value differences and ensures a more balanced selection process.

$$P_i = \frac{2 \times (\text{Rank of Individual})}{N \times (N + 1)}$$

where N is the total number of individuals.

Rank-based selection

Chromosome#	Fitness	Rank	Probabilities	Range
F	84			
A	50			
D	42			
C	36			
B	25			
E	16			

Rank-based selection

Maximization Problem:

6 for highest rank

1 for lowest rank

Chromosome#	Fitness	Rank	Probabilities	Range
F	84	6	0.29	0.77-1
A	50	5	0.24	0.53-0.77
D	42	4	0.19	0.34-0.53
C	36	3	0.14	0.2-0.34
B	25	2	0.10	0.05-0.14
E	16	1	0.05	0-0.05

Rank-based selection-Method 2

- Rank-based selection assigns selection probabilities to individuals based on their rank rather than their raw fitness values.
- **Advantage:** This helps to avoid excessive selection pressure caused by large fitness value differences and ensures a more balanced selection process.
- $P_i = (\text{rank of } i) / \text{Sum of ranks}$

Rank-based selection

Chromosome#	Fitness	Rank	Probabilities	Range
F	84			
A	50			
D	42			
C	36			
B	25			
E	16			

Rank-based selection

Maximization Problem:

6 for highest rank

1 for lowest rank

Chromosome#	Fitness	Rank	Probabilities	Range
F	84	6	0.29	0.72-1
A	50	5	0.24	0.48-0.72
D	42	4	0.19	0.29-0.48
C	36	3	0.14	0.15-0.29
B	25	2	0.10	0.05-0.15
E	16	1	0.05	0-0.05
sum		21		

Rank-based selection

- **What about chromosome with same fitness values?**

What about chromosome with same fitness values

Rank-based selection-Method 1

$$P_i = \frac{2 \times (\text{Rank of Individual})}{N \times (N + 1)}$$

where N is the total number of individuals.

Chromosome#	Fitness	Rank	Probabilities	Range
F	84	5	0.24	0.53-0.76
A	50	4	0.19	0.34-0.53
D	42	3	0.14	0.2-0.34
C	36	2	0.10	0.1-0.2
B	25	1	0.05	0.05-0.1
E	25	1	0.05	0-0.05
Sum			0.76	

What about chromosome with same fitness values

Rank-based selection-Method 2

$$P_i = (\text{rank of } i) / \text{Sum of ranks}$$

Chromosome #	Fitness	Rank	Probabilities	Range
F	84	5	0.31	0.69-1
A	50	4	0.25	0.44-0.69
D	42	3	0.19	0.25-0.44
C	36	2	0.13	0.12-0.25
B	25	1	0.06	0.06-0.12
E	25	1	0.06	0-0.06
Sum			1	

Random selection

- Random selection selects individuals randomly from the population without considering their fitness values.
- Each individual has an equal chance of being selected, regardless of its quality as a solution.



Steady state selection

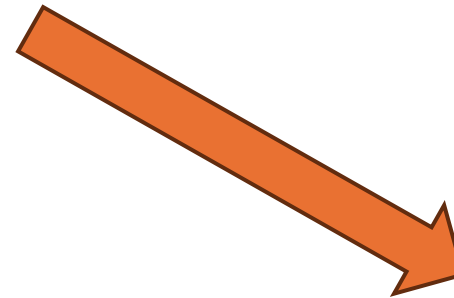
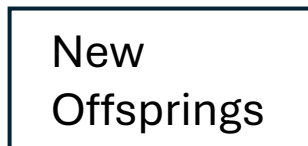
In this method, a few good chromosomes are used for creating new offspring in every iteration.



Population



Then some bad chromosomes are removed, and the new offspring is placed in their places



The rest of population migrates to the next generation without going through the selection process



Generation Gap (GP)

- Number between 0 and 1
- It represents the fraction of population that gets replaced by offspring.
- If $GP = 1$, the new generation consists entirely of new individuals.
- For $GP = 0.4$, $0.4 * P$ offsprings are created and $(1 - 0.4) * P$ individual from the last generation are preserved for the next generation.

Elitism-Selection of k Individuals

- Elitism ensures that a fraction of best individuals from the current generation survives unchanged into the next generation.
- Number between 0 and 1
- If $\text{Pop_size} = 20$ and $\text{Elitism} = 0.1$ then 2 individuals are preserved and 18 may get replaced by offsprings. If $\text{GP} = 0.9$ then these 18 will definitely get replaced.
- Purpose:
 - To retain high-quality solutions across generations.
 - Prevents the loss of the best solutions due to stochastic variation.

Exercise

- In genetic algorithm, a generation has 50 chromosomes, out of which 5 fittest ones are placed in the next generation without any competition from the offspring. The rest of the 45 chromosomes of the next generation are chosen from the pool of parents and offspring. Suppose 30 chromosomes of the offspring make it to the new generation. What is the generation gap and what is the elitism value. Duplicates are not allowed.

Duplicates

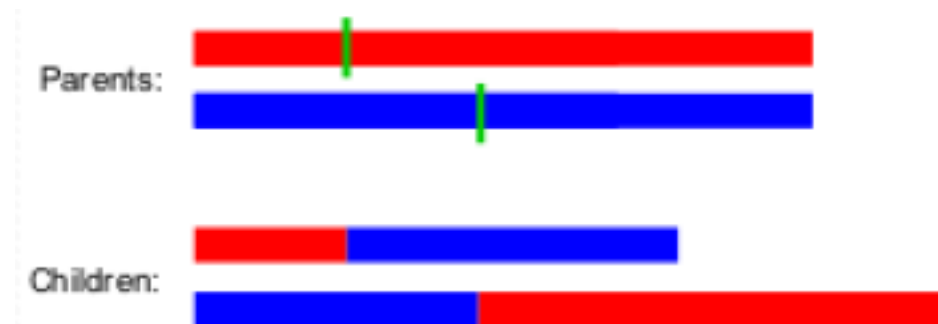
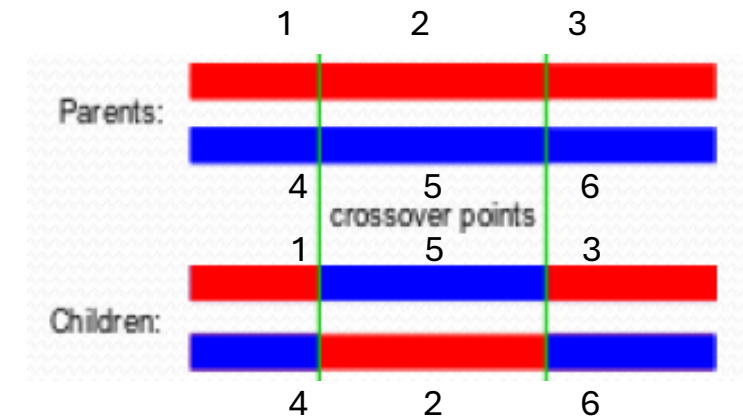
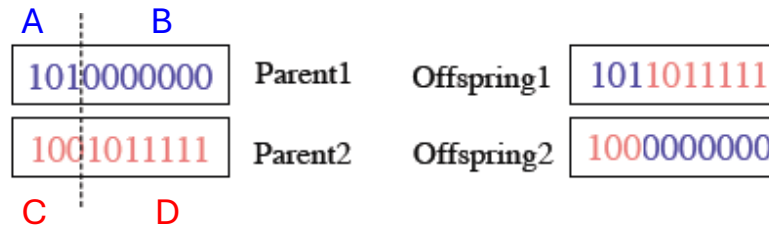
- Eliminating duplicates increases the efficiency of the genetic search and reduces the danger of premature convergence
- Eliminating duplicates means that if an offspring is created which is a duplicate of a chromosome of the current population, we terminate it immediately and create a new one. It increases the processing time in large populations

Crossover Operator- Recombination

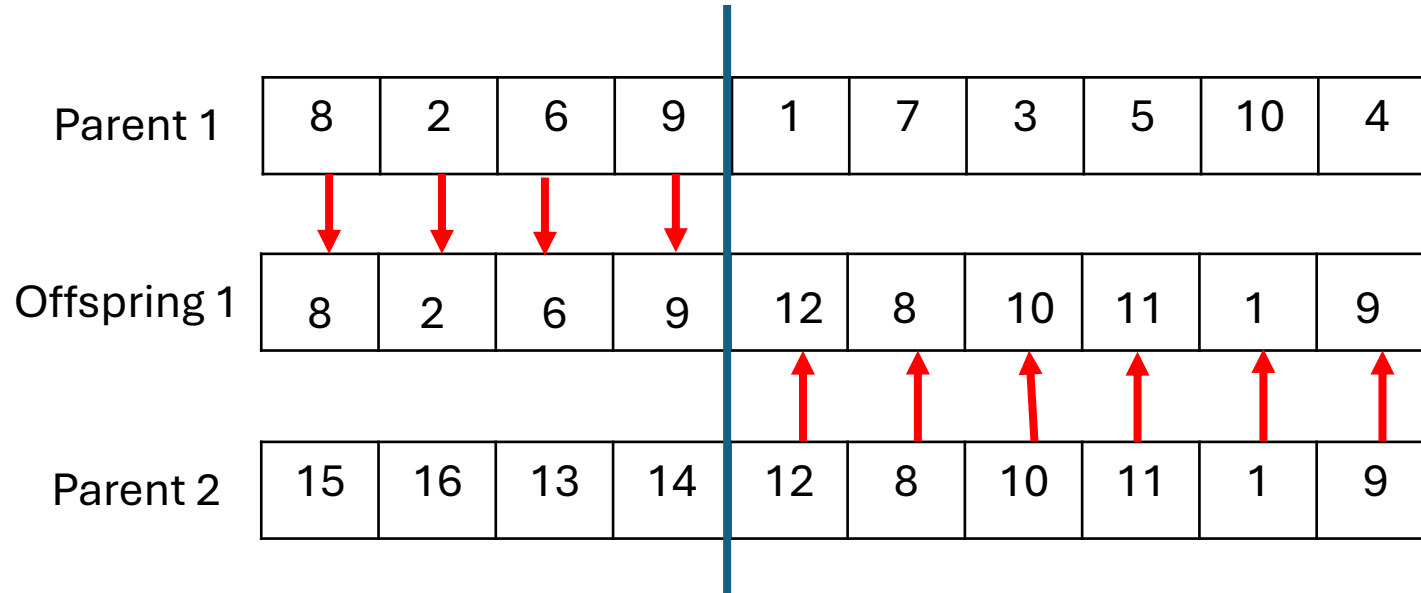
- Accelerates Early Search
- Effective Combination of Subsolutions
- Random Pair Selection
- Encoding-Dependent Crossover Methods
 - Examples include:
 - Single Point Crossover
 - Two Point Crossover

Crossover Operator- Recombination

- Single Point Crossover
- Two Point Crossover
- Others like Cut and paste etc.

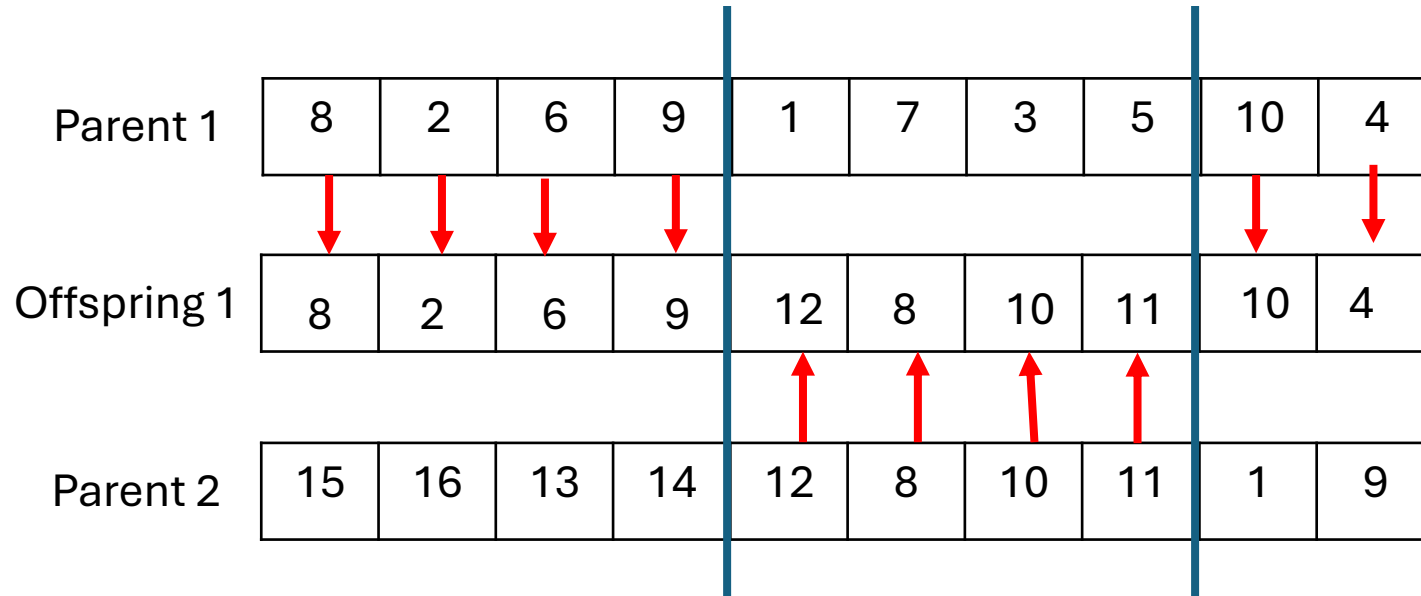


Crossover operator



One point Crossover Operator

Crossover operator



Two point Crossover Operator

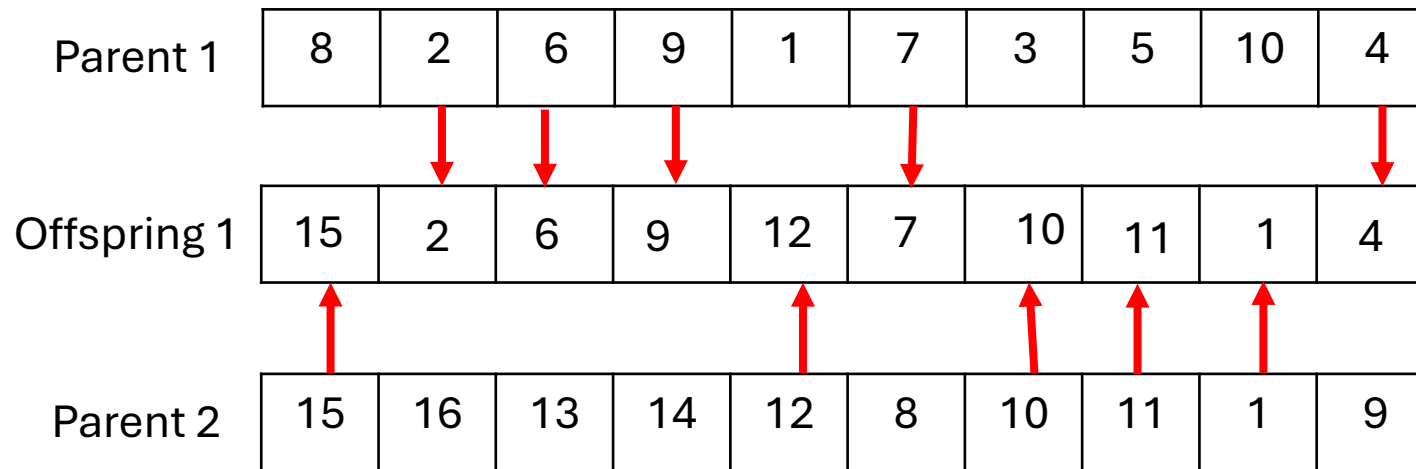
Crossover operator

For each gene position i :

$$\text{Child}[i] = \begin{cases} \text{Parent1}[i] & \text{with probability } p \\ \text{Parent2}[i] & \text{with probability } 1 - p \end{cases}$$

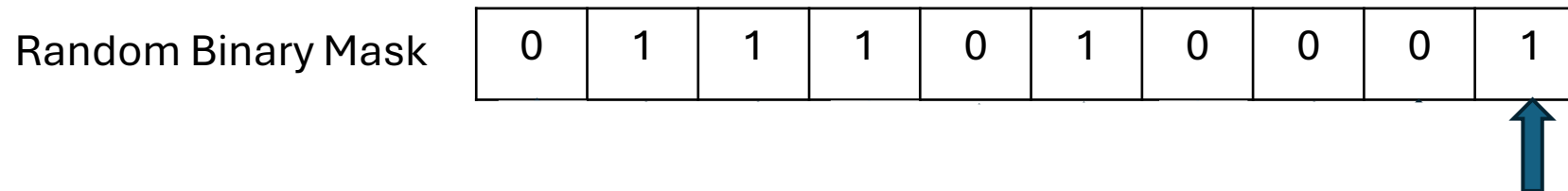
Typically, $p = 0.5$.

A random binary mask determines gene inheritance.



Uniform Crossover Operator

**If mask = 1 → take from Parent 1;
if 0 → Parent 2**



Mutation

- Bit Flip.

Chromosome A

0	1	1	0	1	0	1
---	---	---	---	---	---	---

Mutated

0	1	0	0	1	1	0
---	---	---	---	---	---	---

- Random Resetting (integers)

3	4	1	5	7	2	6
---	---	---	---	---	---	---

3	7	1	5	1	3	6
---	---	---	---	---	---	---

- Swap

3	4	1	5	7	2	6
---	---	---	---	---	---	---

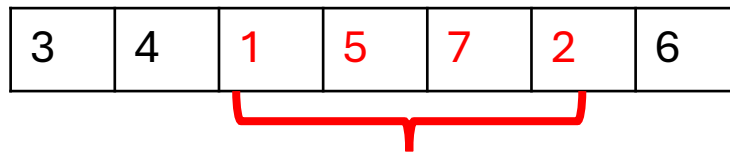


4	3	1	5	6	3	7
---	---	---	---	---	---	---

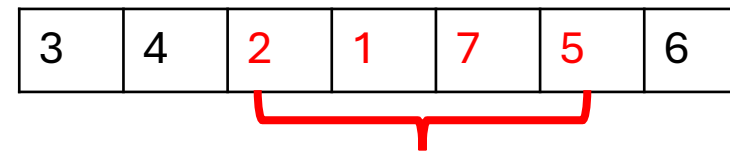


Mutation

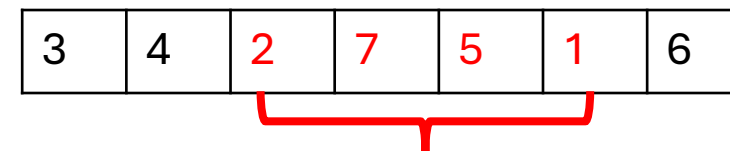
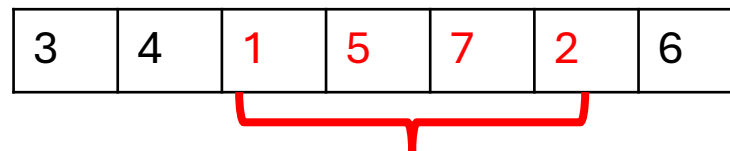
- Scramble



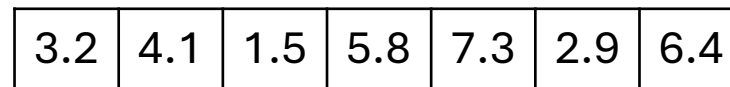
Mutated



- Inversion



- Boundary-Replaces



Termination

- After the reproduction steps we may decide to terminate if:
 - Fitness Threshold
 - Generational Limit
- The resultant output is the optimal solution.

GA

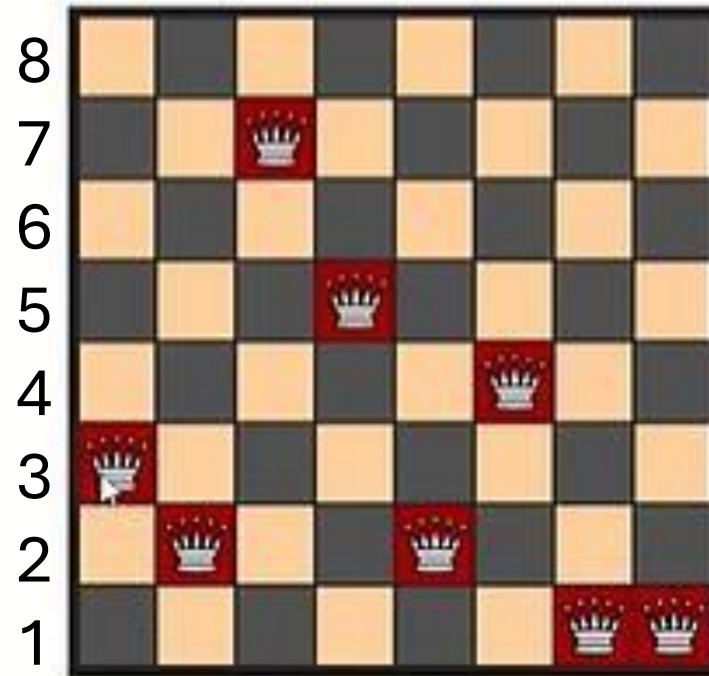
- Step 1: Representing individuals.
- Step 2: Generating a random initial Population
- Step 3: Apply a Fitness Function.
- Step 4: Selecting parents
- Step 5: Crossover of parents to produce new generation.
- Step 6: Mutation of new generation to bring diversity.
- Step 7: Repeat until solution is reached.

8 Queens Problem-GA

Step 1: Representing individuals.

- ▶ Formulate an appropriate method to represent individuals of a population.
- ▶ Array.
- ▶ Index: Column.
- ▶ Value: Row.

3	2	7	5	2	4	1	1
---	---	---	---	---	---	---	---



Step 2: Generating a random initial Population

► Generate random arrangements of 8 queens on a standard chess board.

A



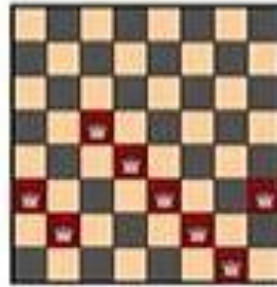
3	2	7	5	2	4	1	1
---	---	---	---	---	---	---	---

B



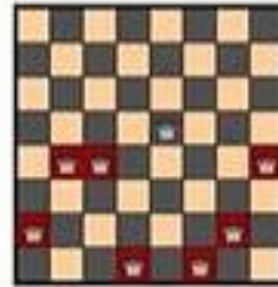
2	4	7	4	8	5	5	2
---	---	---	---	---	---	---	---

C



3	2	5	4	3	2	1	3
---	---	---	---	---	---	---	---

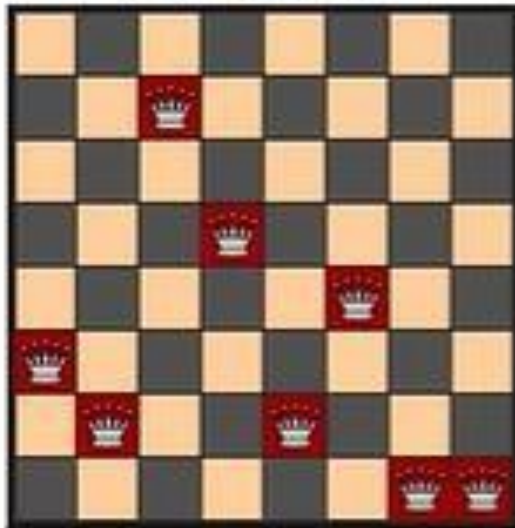
D



2	4	4	1	5	1	2	4
---	---	---	---	---	---	---	---

Step 3: Apply a Fitness Function.

Individual



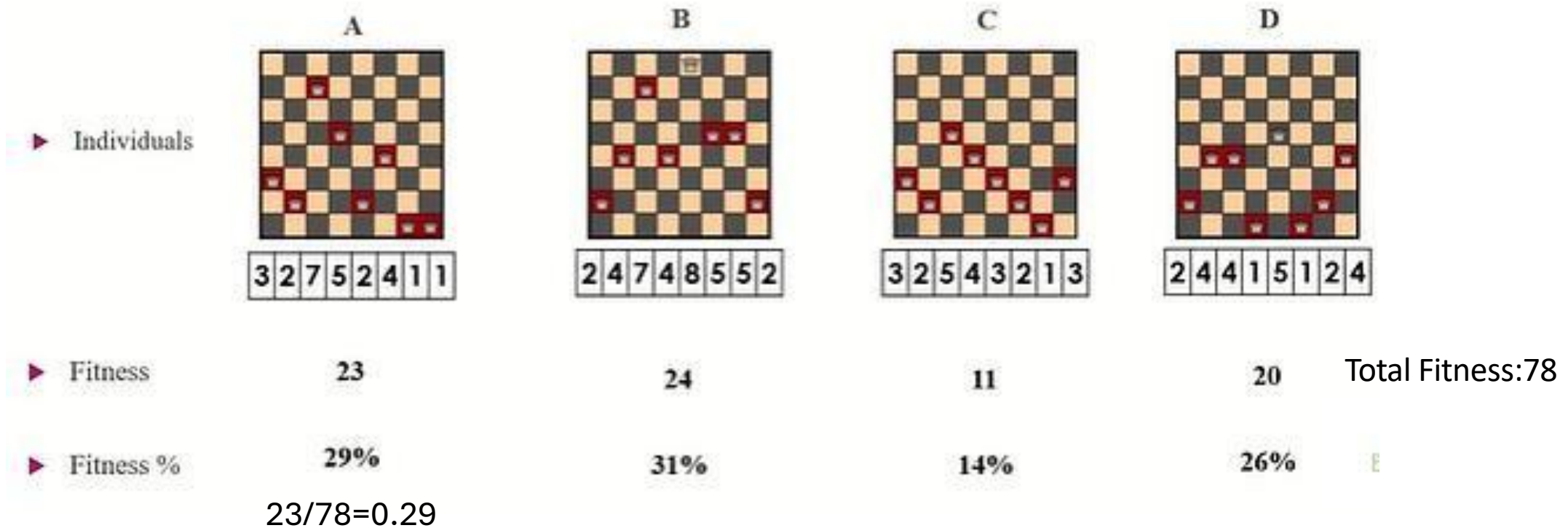
3	2	7	5	2	4	1	1
---	---	---	---	---	---	---	---

Fitness = No. of non attacking pairs

- ▶ Queen 1: 6
- ▶ Queen 2: 5
- ▶ Queen 3: 4
- ▶ Queen 4: 3
- ▶ Queen 5: 3
- ▶ Queen 6: 2
- ▶ Queen 7: 0
- ▶ Queen 8: 0

Total 23

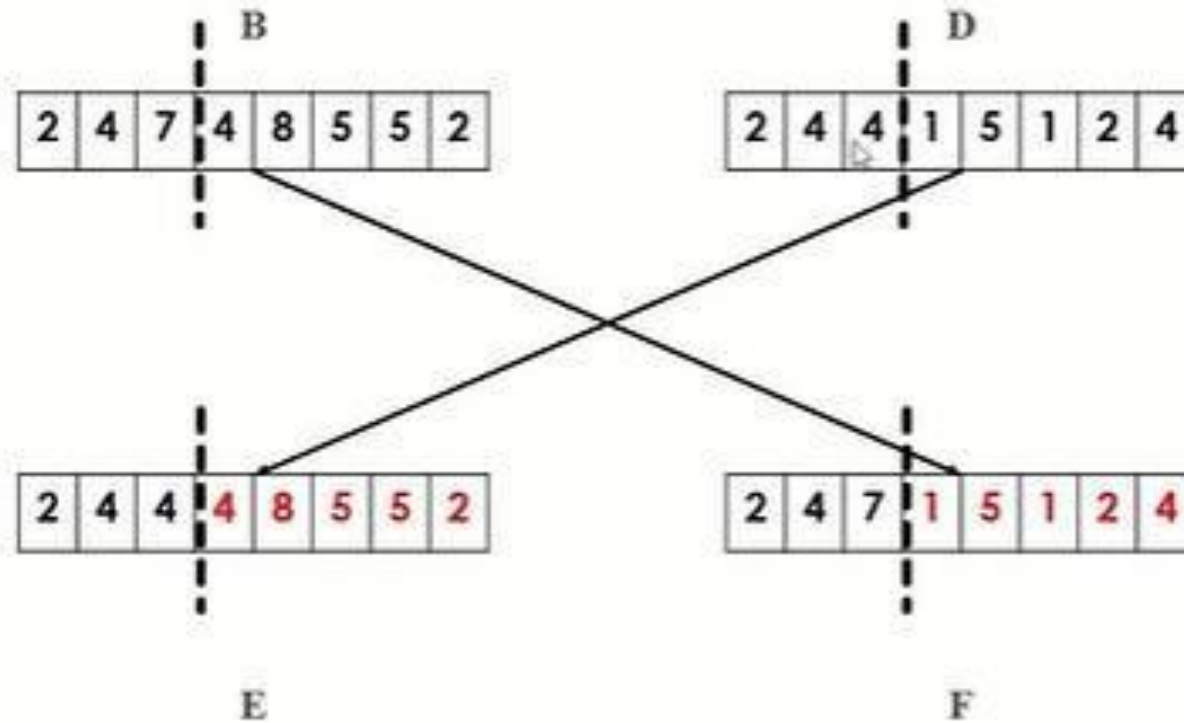
Step 3: Apply a Fitness Function.



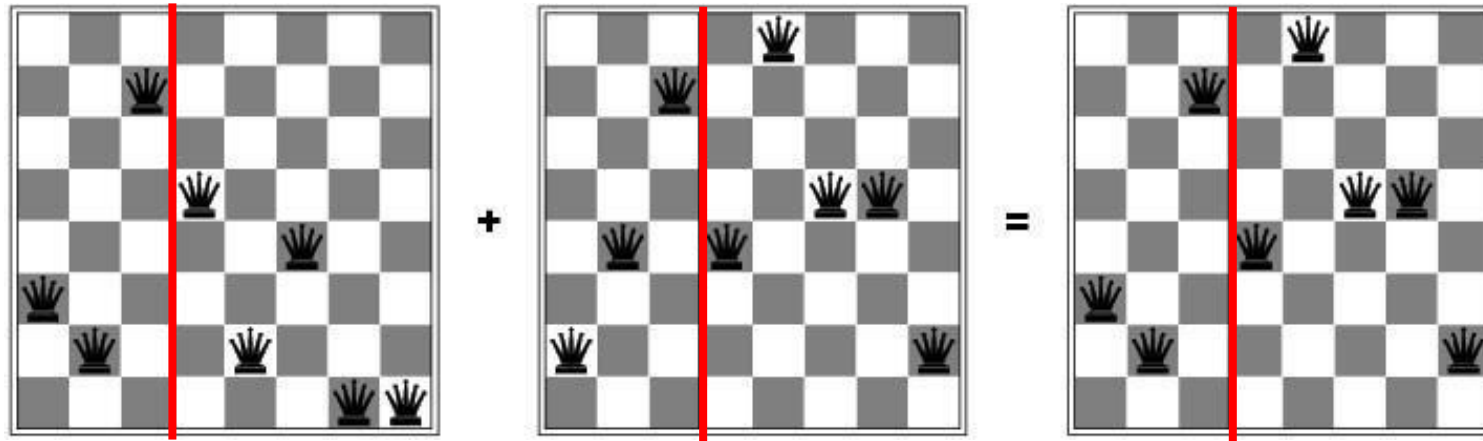
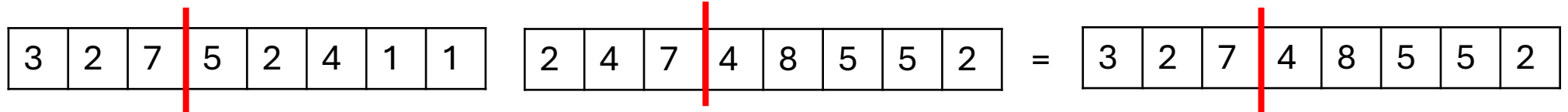
Step 4: Selecting parents

- Use any selection schemes
 - Tournament selection
 - Roulette wheel selection
 - Rank-based selection
 - Random selection
- Suppose B and D are selected

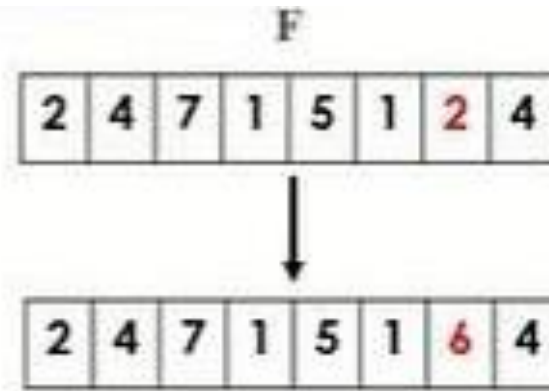
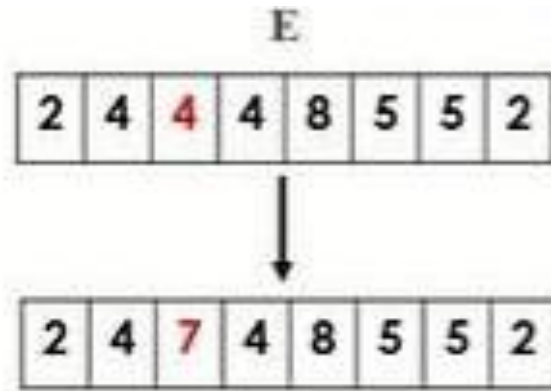
Step 5: Crossover of parents to produce new generation.



Crossover operator



Step 6: Mutation of new generation to bring diversity.



Step 7: Repeat until solution is reached.

- ▶ All steps are repeated until best solution is reached.
- ▶ Best solution = Highest fitness score (28 in this case).

Example: Knapsack Problem (Capacity=12kg)

Item	Weight (kg)	Value
A	5	10
B	3	5
C	7	9
D	2	8

Problem is that which item should be kept in the knapsack so as it will maximize knapsack value without breaking knapsack

Example: Knapsack Problem (Capacity=12kg)

*0 represents absence of item
*1 represent the presence of item
State Space= 2^4

Population

Fitness

Selection

Roulette Wheel selection: the state that has the highest fitness value gets larger share of the wheel

	A	B	C	D
C1	0	1	1	0
C2	0	0	1	1
C3	0	1	1	1
C4	1	0	1	0

Fitness Function for 1:
Weight=3+ 7=10
Value=5+9=14

Fitness Function for 2:
Weight=7+ 2=9
Value=8+9=17

Fitness Function for 3:
Weight=3+ 7+2=12
Value=5+9+8=22

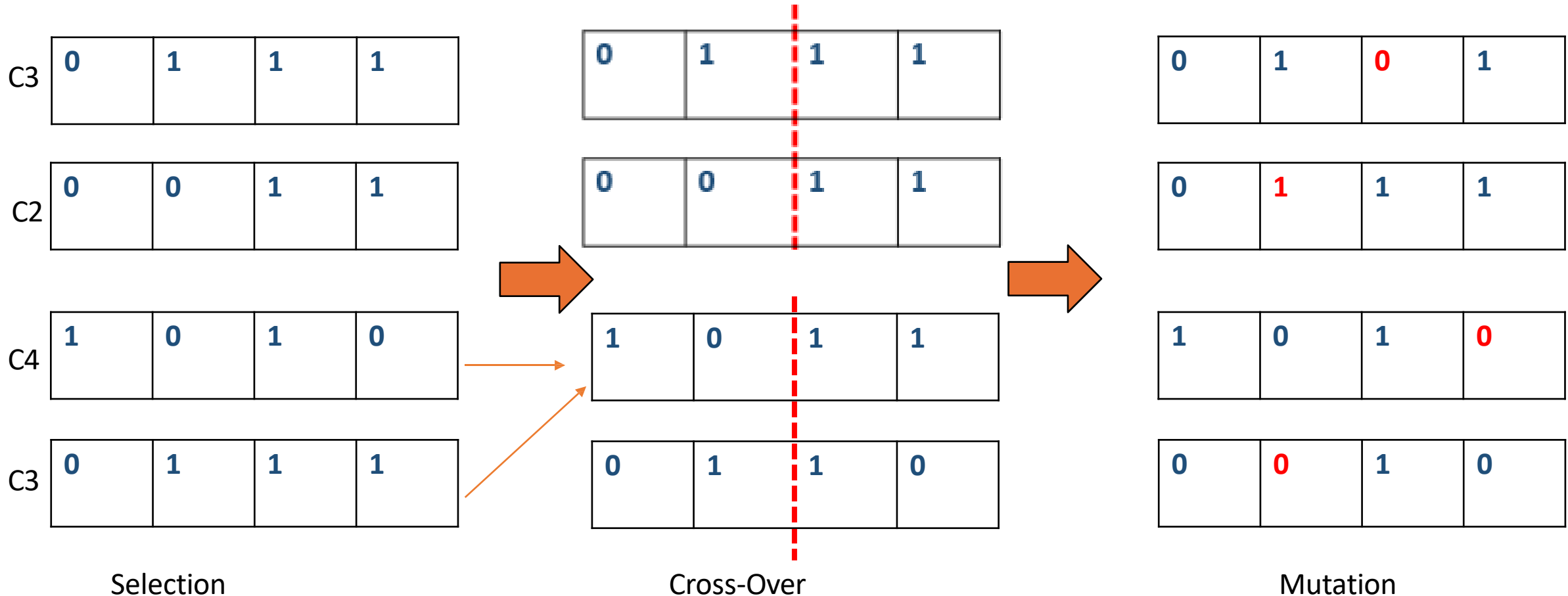
Fitness Function for 4:
Weight=5+ 7=12
Value=10+9=19

0	1	1	1	C3
0	0	1	1	C2
1	0	1	0	C4
0	1	1	1	C3

Example: Knapsack Problem (Capacity=12kg)

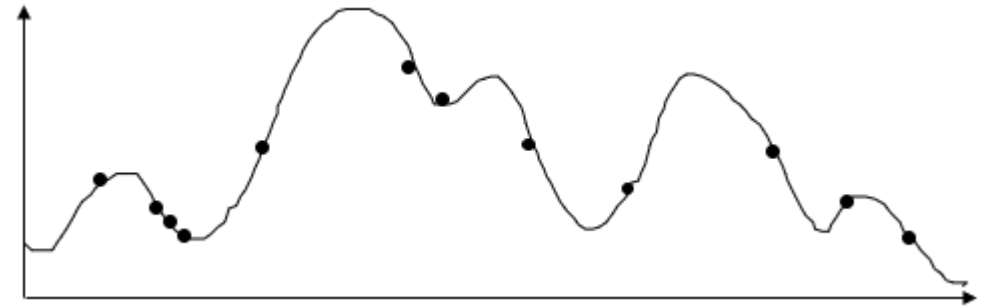
*0 represents absence of item
*1 represent the presence of item
State Space= 2^4

Crossover:
Randomly select
the position around
which gene would
be exchanged

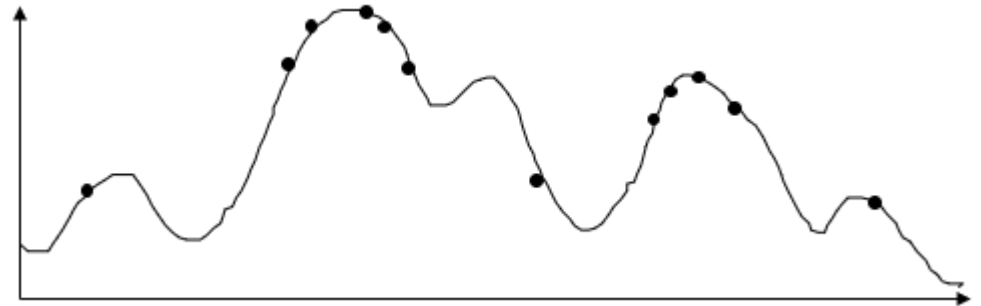


An Abstract Example

- Distribution of Individuals in Generation 0



- Distribution of Individuals in Generation N



Exercise

Problem:

Use a simple Genetic Algorithm to maximize the function:

$$f(x) = x^2$$

where x is an integer in the range $[0, 15]$.

- Use the following settings:
 - Binary encoding (4 bits)
 - Population size = 4
 - Initial population:
 - 0101
 - 1100
 - 0011
 - 1010
 - Selection method: Roulette Wheel Selection
 - Crossover: Single-point crossover (after 2nd bit)
 - Mutation rate: 1 bit at a time

Exercise



Problem:

Use a simple Genetic Algorithm to maximize the function:

$$f(x) = x^2$$

where x is an integer in the range $[0, 15]$.

Chromosome #	Chromosome	decimal (x)	Fitness = F(x)	Probability
A	0101			
B	1100			
C	0011			
D	1010			

Exercise

Problem:

Use a simple Genetic Algorithm to maximize the function:

$$f(x) = x^2$$

where x is an integer in the range $[0, 15]$.

Chromosome #	Chromosome	decimal (x)	Fitness = F(x)	Probability
A	0101	5	25	0.09
B	1100	12	144	0.52
C	0011	3	9	0.03
D	1010	10	100	0.36
Sum			278	

Suppose B and C selected using RW

Exercise

Problem:

Use a simple Genetic Algorithm to maximize the function:

$$f(x) = x^2$$

where x is an integer in the range $[0, 15]$.

Chromosome #	Chromosome	decimal (x)	Fitness = F(x)	Probability
A	0101	5	25	
B	1100	12	144	
C	0011	3	9	
D	1010	10	100	
E	1110	14	196	
F	0100	4	16	

Suppose B and C selected using RW. They are crossed over to produce E & F

Exercise

Problem:

Use a simple Genetic Algorithm to maximize the function:

$$f(x) = x^2$$

where x is an integer in the range $[0, 15]$.

Chromosome #	Chromosome	decimal (x)	Fitness = F(x)	Probability
A	0101	5	25	
B	1100	12	144	
D	1010	10	100	
E	1110	14	196	

Types of Genetic Algorithms

- **Generational GA (Standard GA):**

- Creates new offspring from the old population using genetic operators.
- Entire new population is generated at once.
- The new population completely replaces the old population for the next generation.

- **Incremental/Steady State GA:**

- Introduces one new member into the population at a time.
- Utilizes a replacement or deletion strategy to decide which existing member will be replaced by the new offspring.

Some GA Application Types

Domain	Application Types
Control	gas pipeline, pole balancing, missile evasion, pursuit
Design	semiconductor layout, aircraft design, keyboard configuration, communication networks
Scheduling	manufacturing, facility scheduling, resource allocation
Robotics	trajectory planning
Machine Learning	designing neural networks, improving classification algorithms, classifier systems
Game Playing	checkers, prisoner's dilemma
Combinatorial Optimization	travelling salesman, routing, bin packing, graph colouring and partitioning

Applications: Genetic algorithms

- DNA Analysis
- Image Processing
- Vehicle Routing
- Neural Network

☐ Solving Difficult Problems

GAs prove to be an efficient tool to provide **usable near-optimal solutions** in a short amount of time. 🟡

☐ Failure of Gradient Based Methods

In most real-world situations we have a very complex problem as it suffer from an inherent tendency of getting stuck at the local optima as shown here.

☐ Getting a Good Solution Fast

Some difficult problems like the Travelling Salesperson Problem (TSP), have real-world applications like path finding and VLSI Design.

Advantages of Genetic Algorithms

Modularity

Robust in
Noisy
Environments

Multi-
Objective
Optimization

Improves
Over Time

Parallelism

Issues for GA Practitioners

1.Implementation Challenges:

Choosing representation, population size, mutation rate, and other parameters.

2.Selection and Deletion Policies:

Deciding how individuals are selected and replaced in each generation.

3.Crossover and Mutation Operators:

Selecting appropriate operators to generate better offspring.

4.Performance and Scalability:

Ensuring the algorithm performs efficiently as the problem size increases.

5.Evaluation Function:

The solution quality heavily depends on the evaluation function, which can be the most challenging part to design.

Performance Measures for Single-Objective Optimization Algorithms

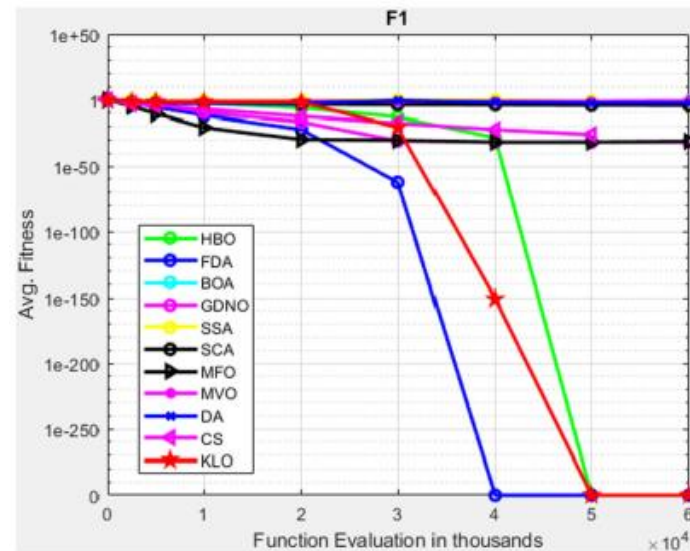
- **Accuracy of the Solution**

- **Objective Value:** The value of the objective function for the best-found solution.
- **Optimality Gap:** The difference between the found solution and the known optimal solution (if available).

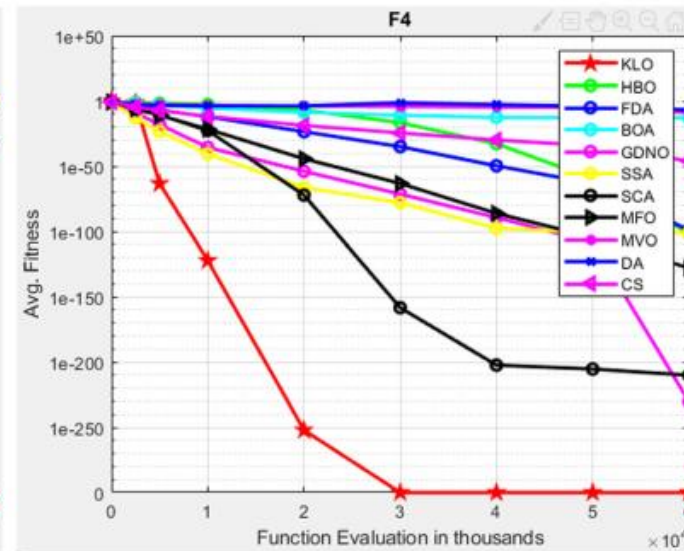
Performance Measures for Single-Objective Optimization Algorithms

- **Convergence**

- **Rate of Convergence:** Measures how quickly the algorithm approaches the optimal solution.
- **Final Convergence:** The closeness of the final solution to the optimal value after the algorithm terminates.



(a) Beale



(b) Matyas

Performance Measures for Single-Objective Optimization Algorithms

- **Computational Efficiency**

- **Execution Time:** The time taken by the algorithm to find a solution.
- **Number of Iterations:** The number of iterations required to converge to the solution.

- **Robustness**

- **Consistency:** The algorithm's ability to produce similar results across multiple runs with different initializations.
- **Sensitivity:** The impact of parameter changes on the algorithm's performance.

Performance Measures for Single-Objective Optimization Algorithms

- **Exploration and Exploitation**

- **Exploration:** The ability to search diverse regions of the solution space.
- **Exploitation:** The ability to refine and improve the current solution in promising regions.

- **Scalability**

- **Problem Size:** The algorithm's ability to handle increasing problem sizes without a significant drop in performance.

Performance Measures for Single-Objective Optimization Algorithms

- **Memory Usage**
 - **Resource Utilization:** The amount of memory and computational resources required during execution.
- **Termination Criteria**
 - **Stopping Conditions:** Evaluating whether the algorithm terminates effectively based on predefined criteria, such as reaching a specific accuracy, maximum iterations, or time limit.
- These measures help in comparing different algorithms and selecting the most suitable one for a given optimization problem.

References

- Engelbrecht Chapter 8 &9
- A.E. Eiben and J.E. Smith Chapter 3,4, 6